RAPPORT SAE2.1

Réalisation du jeu « SameGame »

Sommaire :

Partie 1 : Introduction du jeu SameGame..... 3

Partie 2 : Description des fonctionnalités..... 4

- Choix et génération de la carte, aléatoire ou définie..... 4
- Détection et surbrillances des groupes/voisins..... 5
- Fonctionnement de la gravité..... 7
- Réorganisation de la grille..... 7
- Affichage du score et progression..... 8

Partie 3 : Structure du programme..... 8

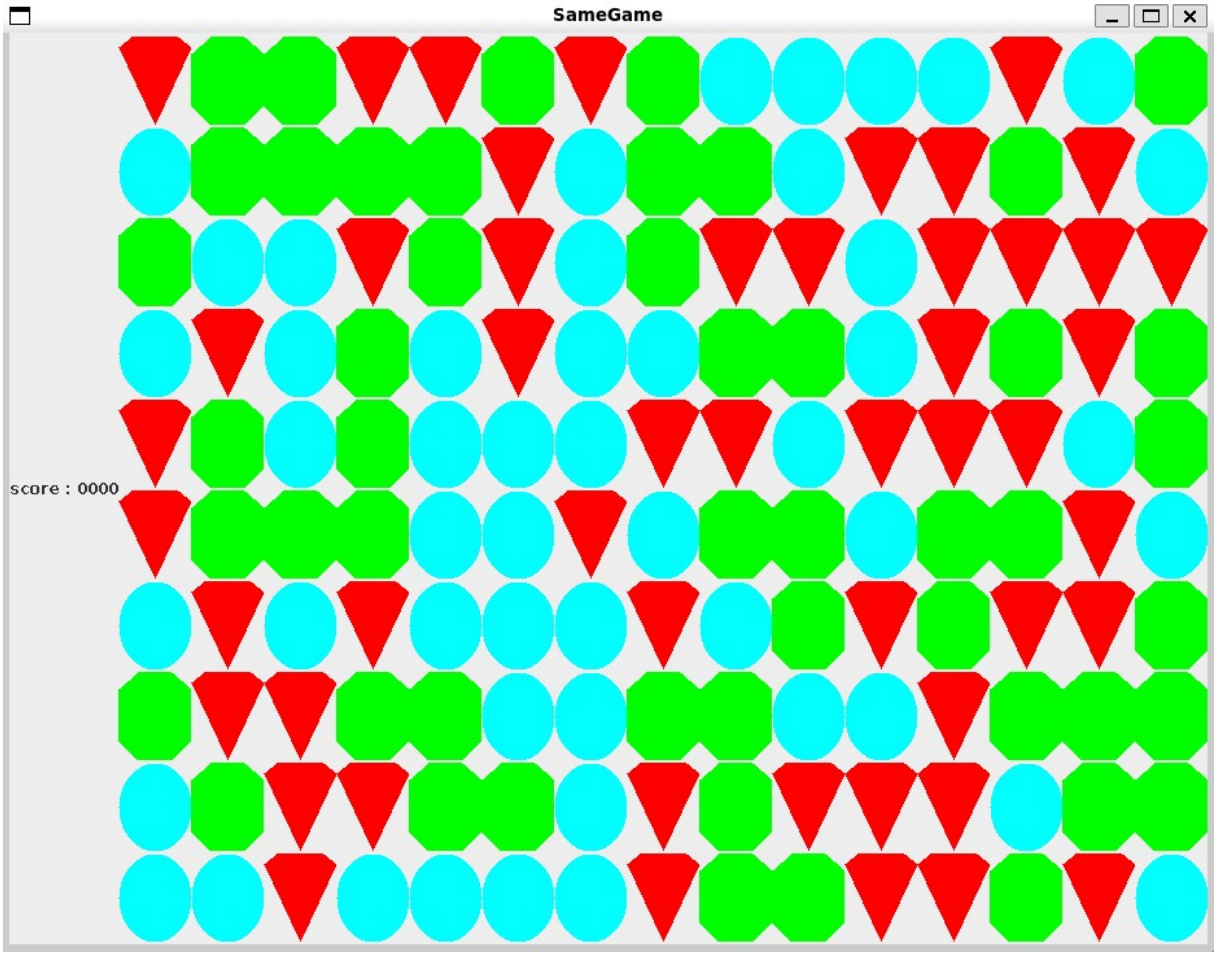
- Fonctionnement et diagramme du menu..... 9
- Fonctionnement et diagramme d'une partie..... 10

Partie 4 : Avis personnels..... 11

- Nicolas Miekisiak.....
- Michel Tran.....

Partie 1 : Introduction du jeu « SameGame »

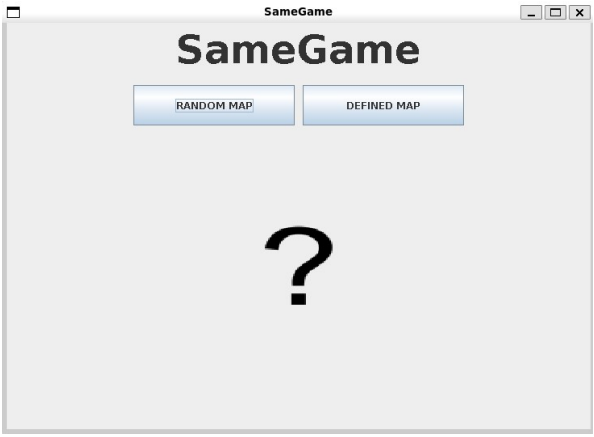
SameGame est un jeu japonais sorti en 1985, le jeu se déroule sur une grille de 10x15, celle-ci est composée de blocs de couleur différentes, rouge, vert ou bleu. Le but est de faire d'augmenter son score le plus haut possible en supprimant des groupes supérieur ou égale à deux blocs adjacents et de même couleur. Plus le groupe contient de bloc plus il rapportera de points. Chaque bloc est soumis à la gravité, et une fois qu'une colonne est entièrement supprimée les colonnes à sa droite sont décalées vers la gauche. Lorsque plus aucun groupe n'est possible, alors la partie se termine.

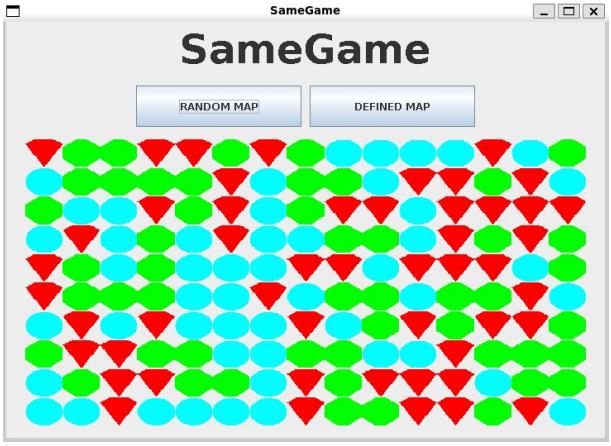


Partie 2 : Description des fonctionnalités

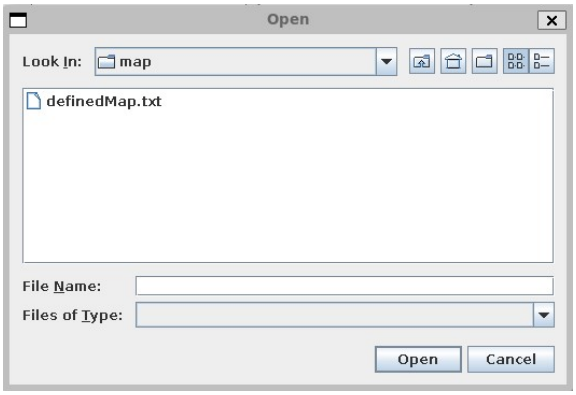
Choix et génération de la carte, aléatoire ou définie :

Tout d'abord le programme se lance sur un menu, nous laissant le choix entre deux boutons permettant de choisir une carte aléatoire ou définie.





Une fois le bouton cliqué, le programme envoie directement l'information. Si le choix de l'utilisateur est « CARTE ALEATOIRE » alors la génération de la carte aléatoire se fera instantanément après. Tandis que, si le choix de l'utilisateur est « CARTE DEFINIE » alors le programme demandera à l'utilisateur de choisir le fichier de la carte qu'il souhaite choisir, ensuite la carte sera générée en fonction du fichier choisi.



Détection et surbrillances des groupes/voisins :

L'algorithme de recherche des voisins se base un peu sur une recherche en largeur, c'est à dire qu'il va regarder un objet ajouter tous ses voisins puis aller dans l'un d'eux rajouter d'autres voisins et retourner dans l'un des autres voisins du premier, etc.

Ainsi, le meilleur outil pour faire cela est un tableau.

La recherche est divisée en trois plus petits algorithmes : getVoisinDirect, contient et getVoisin.

getVoisinDirect :

Cette méthode s'occupe d'aller chercher les voisins directs de chaque objet de la grille il effectue deux boucles une qui regarde les voisins verticaux et une autre qui regarde les horizontaux, chaque voisin doit donc respecter plusieurs conditions pour être ajouter au tableau :

- Ses coordonnées doivent être dans la grille.
- Il doit exister et être vivant c'est à dire pas null et avec un état à true.
- Il doit avoir la même couleur que l'objet de base.

La méthode renvoie finalement le tableau des voisins directs.

contient :

Ce programme prend en argument un tableau d'ObjetGrille et un ObjetGrille ; il renvoie "true" si l'objet est dans le tableau "false" sinon.

Le programme parcourt seulement le tableau en vérifiant à chaque fois si les objets sont identiques.

getVoisin :

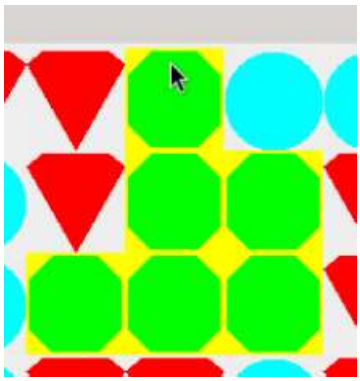
Cette méthode prend en argument une matrice d'ObjetGrille, une ligne et une colonne et va renvoyer le groupe de voisins de l'ObjetGrille (ligne, colonne).

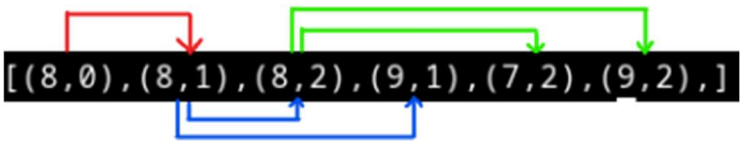
Tout d'abord il créé un tableau allVoisin en y ajoutant l'objet de départ puis démarre une boucle qui s'arrêtera quand la capacité du tableau (150 car il y a 150 voisins maximum) sera excédée ou quand le l'objet du tableau est null : donc qu'on est passé par tous les voisins. On initie un tableau temporaire tempVoisin qui va chercher les voisins directs et les stockés.

Dans cette boucle on démarre une autre boucle mais parcourant tempVoisin pour vérifier si les derniers voisins trouvés sont déjà dans le tableau allVoisin si ce n'est pas le cas on les rajoute.

Finalement on renvoie le tableau allVoisin.

Exemple : la souris se situe en (8,0) sur la grille :



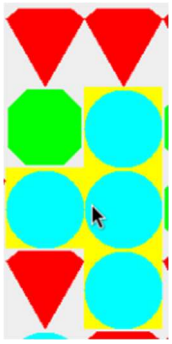


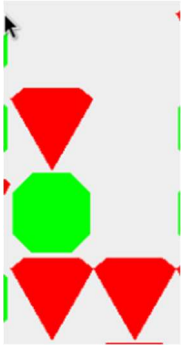
Fonctionnement de la gravité :

Pour simuler la gravité, on récupère d'abord un tableau contenant les colonnes affectées par la dernière destruction car c'est le seul endroit où l'on doit faire tomber des objets de notre grille, c'est la fonction getColCasse() qui s'occupe renvoyer ce tableau.

Ensuite on parcourt ce tableau et pour chaque colonne on démarre de tout en bas puis on remonte jusqu'à trouver un ObjetGrille ayant un état à false : alors une autre boucle se lance pour trouver le premier ObjetGrille vivant au-dessus et intervertir les deux objets ; ensuite on recommence à monter.

Tout cela simule finalement la gravité :

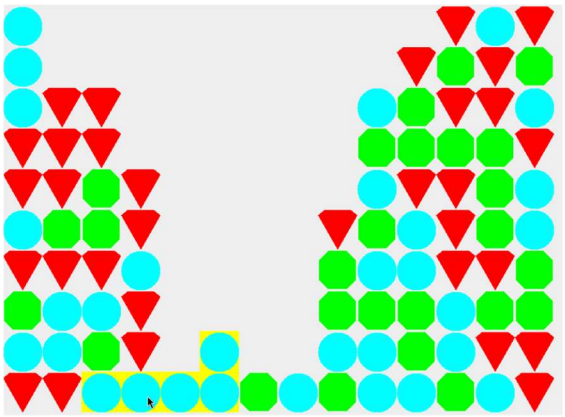


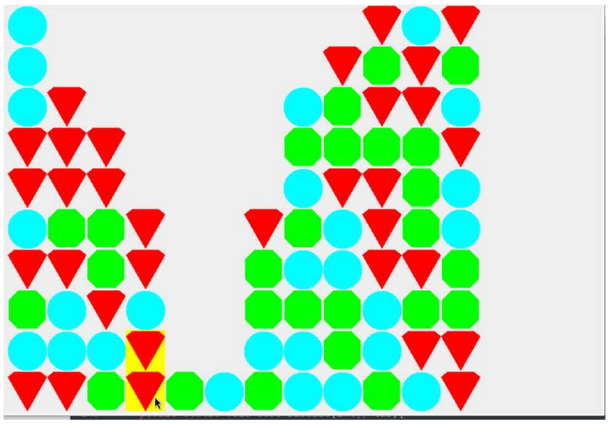


Réorganisation de la grille :

Pour réorganiser la grille, c'est à dire la coller vers la gauche lorsqu'une colonne est vidé, on utilise encore une fois la liste des colonnes cassées car c'est seulement sur celles-ci qu'une colonne peut devenir vide.

Cependant cette fois-ci on doit la trier dans l'ordre décroissant pour que le programme fonctionne correctement car on va parcourir chaque colonne affectée de droite vers la gauche (vue que le tableau est décroissant) et si elle est vide on bouge toutes la partie à sa droite d'un cran vers la gauche :





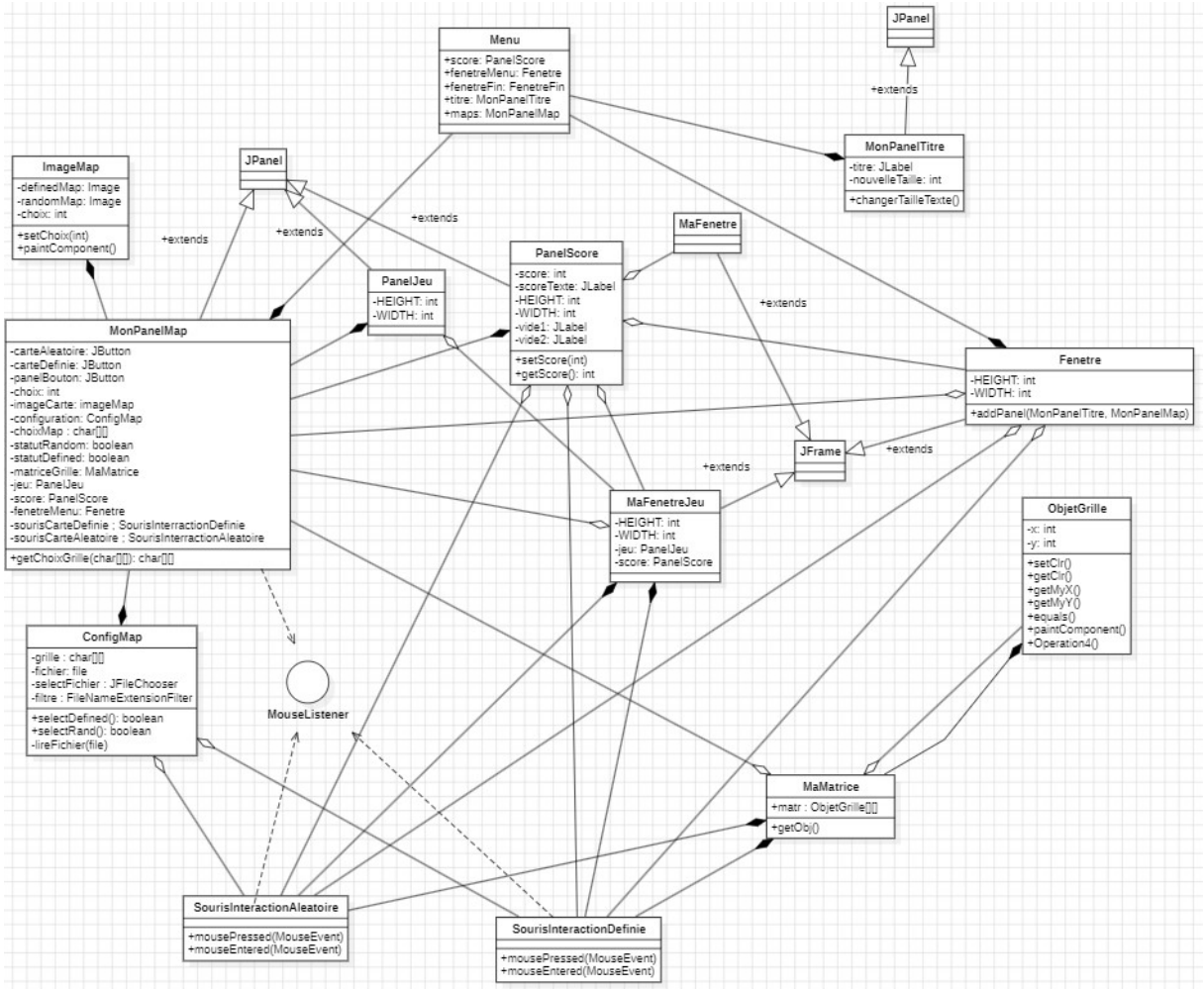
Affichage du score et progression :

L'affichage du score se fait sur le côté gauche de la fenêtre du jeu, le score s'actualise à chaque fois qu'un groupe est supprimé., grâce à la méthode setScore(). La formule de progression du score est : (nombre de voisins – 2) ^{2}

Partie 3 : Structure du programme

Fonctionnement et diagramme du menu

Voici à quoi ressemble le diagramme de classe des classes qui permettent le bon fonctionnement de l'affichage, gestion du menu et la création la carte.



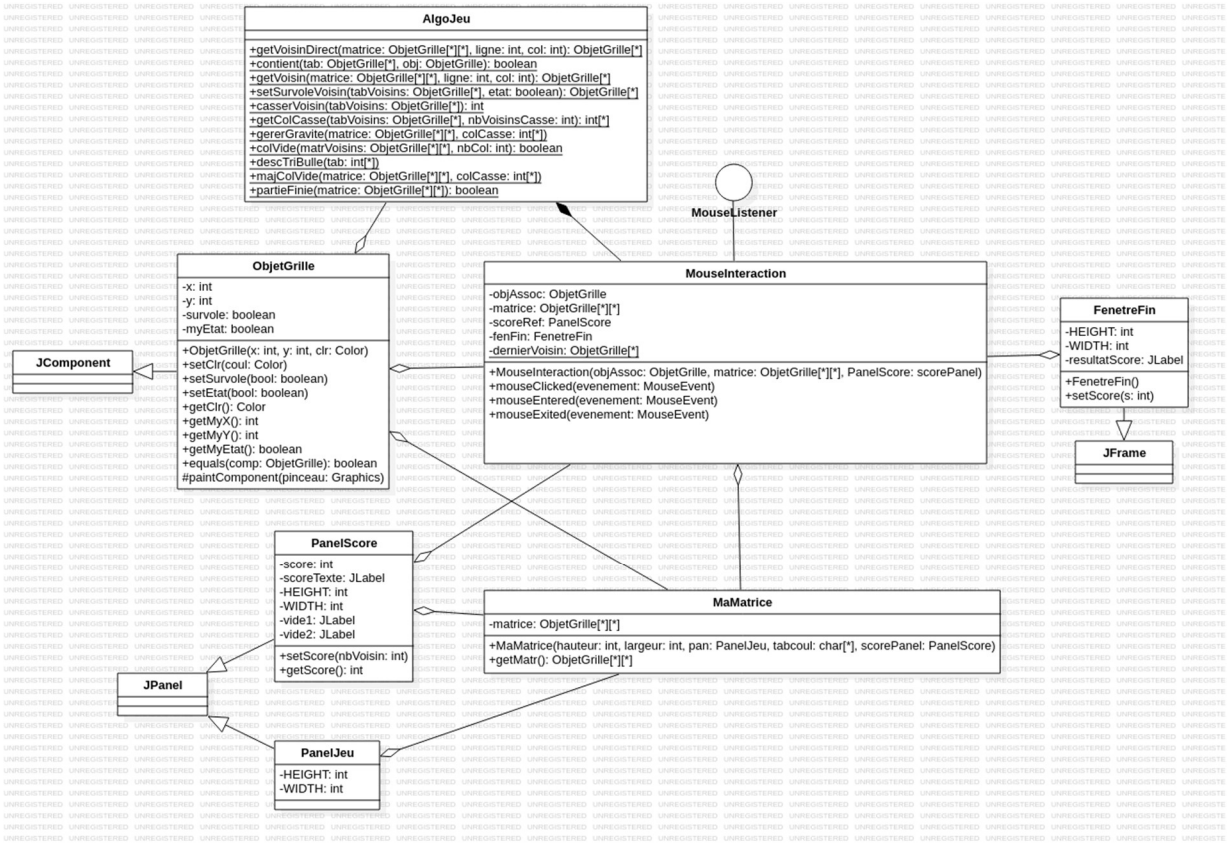
La classe Menu s'occupe de l'affichage des différents de ces trois différentes classes, elle gère trois classes, MonPanelMap, MonPanelTitre et Fenetre.

Les classes MonPanelMap et MonPanelTitre s'occupent d'afficher le contenu du menu, MonPanelMap va recevoir l'information

donnée par l'utilisateur. Pour ensuite créer la carte correspondante, Grace la classe MaMatrice et ConfigMap. Par la suite une fenêtre du jeu va s'afficher grâce à la classe MaFenetreJeu, qui va regrouper MonPanelScore et MonPanelJeu.

Fonctionnement et diagramme d'une partie

Voici à quoi ressemble le diagramme de classe qui contient les classes qui permettent le déroulement du jeu.



Partie 4 : Avis personnels

Nicolas Miekisiak :

Ce projet m'a énormément apporté, déjà car j'aime beaucoup réfléchir aux algorithmes derrière les jeux en général, ainsi devoir les refaire m'a beaucoup plu.

De plus, faire ce projet a consolidé mes connaissances et m'en a créé des nouvelles par exemple je n'avais jamais fait de Makefile pour un code en Java auparavant et je n'avais pas non plus manipulé de tableaux multidimensionnels dans ce langage même si ça reste très proche du C j'ai beaucoup aimé refaire toutes ces choses-là.

Michel Tran :

J'ai bien aimé la réalisation du projet dans l'ensemble, avant j'avais du mal avec les événements et les fichiers, et grâce au projet cela m'a permis de renforcer mes lacunes. J'ai aussi apprécié le jeu en lui-même, je ne le connaissais pas et c'était une bonne découverte, mon record de points est de 4000pts sur une carte aléatoire.

Si je devais retenir quelque chose de projet, c'est de ne pas réaliser une SAE en même temps que de travailler à côté. Durant cette semaine de SAE, je travaillais la journée et en fin de journée m'occupais de la SAE, ce n'était pas une expérience très agréable.